PUC Minas



Algoritmos e Estruturas de Dados I

Prof. Dr. João Paulo Aramuni

Aula 02

Operações sobre dados e estruturas de controle

AEDS I - Manhã

02

Operações sobre dados e estruturas de controle

Unidade de Ensino

2.1 Armazenamento, constantes e variáveis

2.2 Transferências, atribuições, entradas e saídas

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

2.4 Estruturas de controle

2.4.1 Estrutura sequencial

2.4.2 Estruturas condicionais: simples, composta e múltipla

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle



2.1 Armazenamento, constantes e variáveis

Em linguagem C, o armazenamento refere-se à alocação e manipulação de dados na **memória** do computador.

Dominar a manipulação de memória RAM (Random Access Memory) é fundamental para entender algoritmos e estruturas de dados em C.

Existem diversos tipos de armazenamento em C, incluindo constantes, variáveis, arrays e ponteiros.



2.1 Armazenamento, constantes e variáveis

Em linguagem C, você pode usar o conceito de **constantes** para representar valores que **não** podem ser alterados durante a execução do programa.

Existem duas formas principais de definir constantes em C: usando a diretiva **#define** e usando a palavra-chave **const**.

Exemplo: **#define** PI 3.14159

- Sua definição não usa ponto-e-vírgula, nem sinal de igualdade.



2.1 Armazenamento, constantes e variáveis

Costuma-se definir o nome das constantes com letras **maiúsculas**, para diferenciá-los de nomes de variáveis e funções.

Essa prática melhora a legibilidade do programa.

Definição de constantes facilita a manutenção do programa.



2.1 Armazenamento, constantes e variáveis

Palavra-chave **const**:

A palavra-chave const é usada para criar variáveis constantes.

Essas variáveis têm seus valores definidos na compilação e não podem ser modificadas durante a execução do programa.

Exemplo: **const int** MAX_NUM = 100;



2.1 Armazenamento, constantes e variáveis

```c
#include <stdio.h>
#include <stdlib.h>

#define PI 3.14159

int main(void) {

    int idade = 26;
    const int MAX_NUM = 100;

    printf("Minha variável int comum = %d\n", idade);
    printf("Minha variável int constante = %d\n", MAX_NUM);
    printf("Minha constante = %f\n", PI);

    MAX_NUM = 99;

    return EXIT_SUCCESS;
}
```

cannot assign to variable 'MAX_NUM' with const-qualified type 'const int'

Console | Problems | Tasks | Properties

CDT Build Console [Prj_Variaveis]

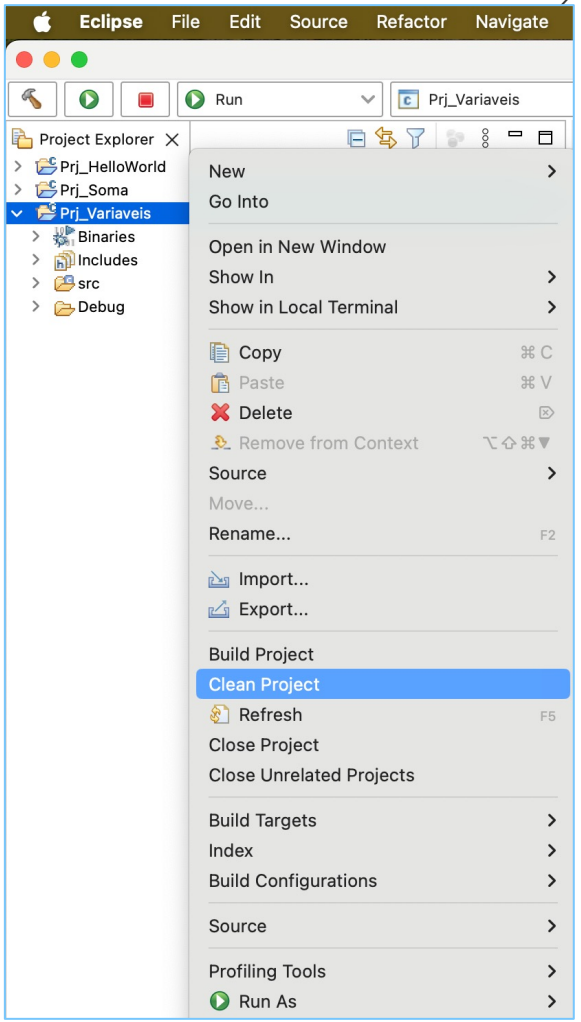
```
../src/Prj_Variaveis.c:25:10: error: cannot assign to variable 'MAX_NUM' with const-qualified type 'const int'
        MAX_NUM = 99;
        ~~~~~~~ ^
../src/Prj_Variaveis.c:19:12: note: variable 'MAX_NUM' declared const here
        const int MAX_NUM = 100;
        ~~~~~~~~~~^~~~~~~~~~~~~
1 error generated.
make: *** [src/Prj_Variaveis.o] Error 1
"make all" terminated with exit code 2. Build might be incomplete.
```

2.1 Armazenamento, constantes e variáveis

Dica: Limpe o projeto com o Clean Project.

Caso algum erro persista, mesmo após corrigi-lo, experimente executar o **Clean Project** para limpar os arquivos binários e logo após faça o **Build Project** para reconstruir estes arquivos.

Feito isso, rode o projeto normalmente com **Run As** > Local C/C++ Application.



2.1 Armazenamento, constantes e variáveis

A escolha entre #define e const muitas vezes depende do contexto e das necessidades específicas do programa.

No entanto, a abordagem usando **const** é geralmente preferida, pois fornece um **tipo** de dado específico e pode ser mais seguro em termos de escopo e tipo.

Além disso, const é mais flexível quando se trata de escopo e pode ser usado para criar constantes locais em funções.



2.1 Armazenamento, constantes e variáveis

As variáveis são **espaços na memória** que armazenam dados com um determinado tipo. Por exemplo, int, float, char, etc.

A declaração de uma variável em C segue o formato: tipo_nomeDaVariavel;, onde "tipo" é o tipo de dado e "nomeDaVariavel" é o nome atribuído à variável.



2.1 Armazenamento, constantes e variáveis

Exemplo:

int idade; // Declaração de uma variável do tipo inteiro chamada "idade" idade = 25; // Atribuição de valor à variável

Variáveis devem ser declaradas **antes de serem usadas**, no início de um bloco de código.

Outros exemplos:

char letra;
float temperatura, pressao;
int dia, mes, ano;



2.1 Armazenamento, constantes e variáveis

Variáveis de tipos diferentes devem ser obrigatoriamente declaradas em linhas diferentes.

Numa mesma linha poderão ser definidas uma ou mais variáveis do mesmo tipo, deve-se separar os nomes das mesmas por vírgulas.

Um dos objetivos de se declarar variáveis no inicio do algoritmo é para que seja reservada (alocada) uma área na memória.

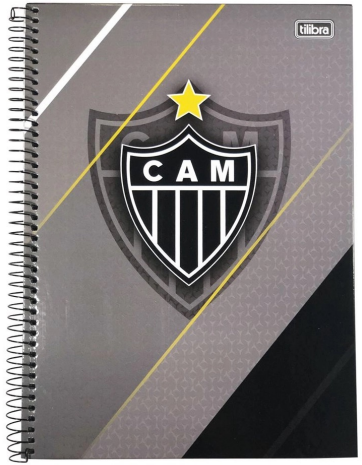


2.1 Armazenamento, constantes e variáveis

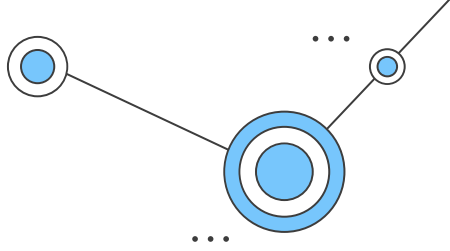
Dica: Compre um caderno.

- Desenhar a memória RAM a lápis te ajudará muito a entender como as variáveis são alocadas e manipuladas. Esta é uma maneira eficaz de internalizar conceitos fundamentais de programação e estruturas de dados em C.

- Ao visualizar e esboçar a disposição das variáveis na memória, você terá uma representação gráfica que pode ajudar a compreender como o espaço de armazenamento é alocado para diferentes tipos de dados, como inteiros, pontos flutuantes e estruturas complexas.

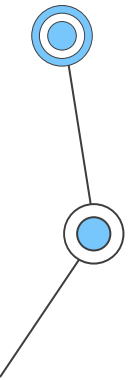


2.1 Armazenamento, constantes e variáveis



Em linguagem C, quando você declara uma variável, ela é armazenada em um endereço de memória.

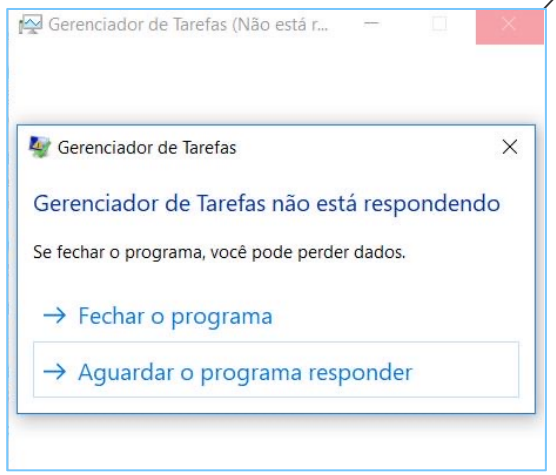
Esse endereço é representado em formato hexadecimal, que é uma base numérica de 16 dígitos. A representação em hexadecimal é comumente usada porque é mais compacta e legível em comparação com a representação em binário.

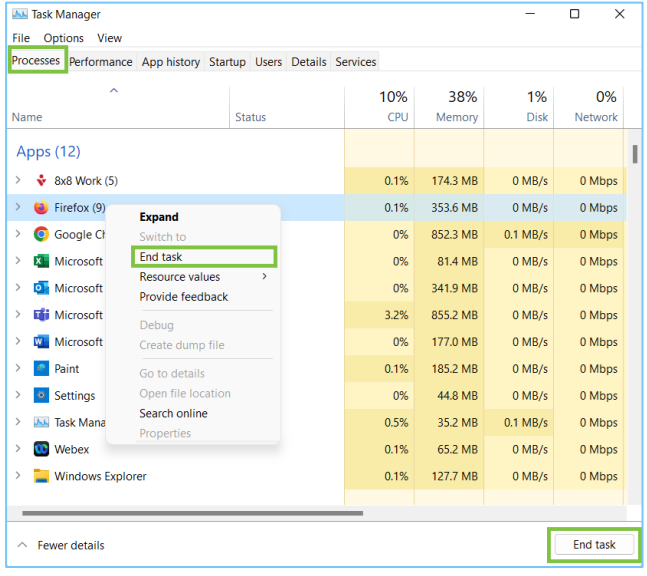


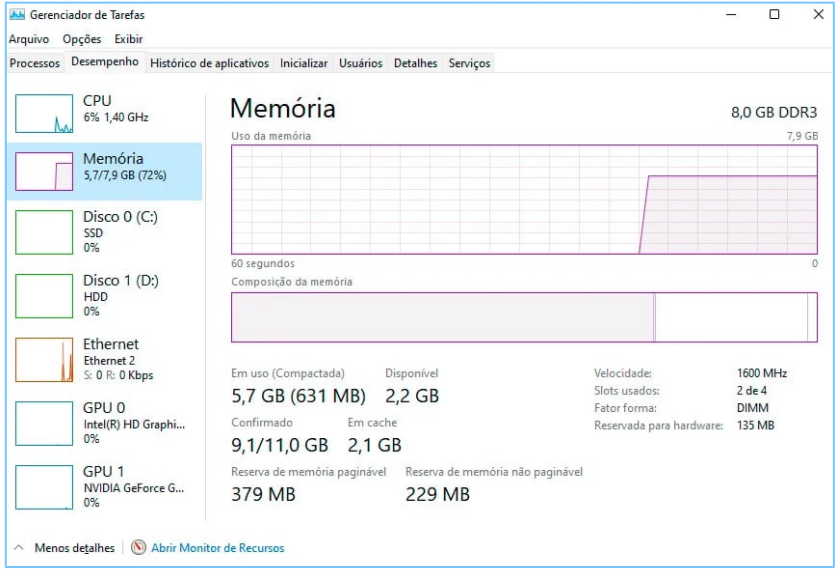
Endereço de memória	Conteúdo (binário)	Valor (decimal)
#0000	00000000	0
#0001	00000001	1
#0002	00000010	2
#0003	00000011	3
#0004	00000100	4
#0005	00000101	5
...	...	...
#FFFF	...	...

2.1 Armazenamento, constantes e variáveis

Por exemplo, se você tem uma variável int chamada numero e ela está armazenada no endereço hexadecimal 0x1000, isso significa que o valor dessa variável está armazenado na posição de memória 0x1000.



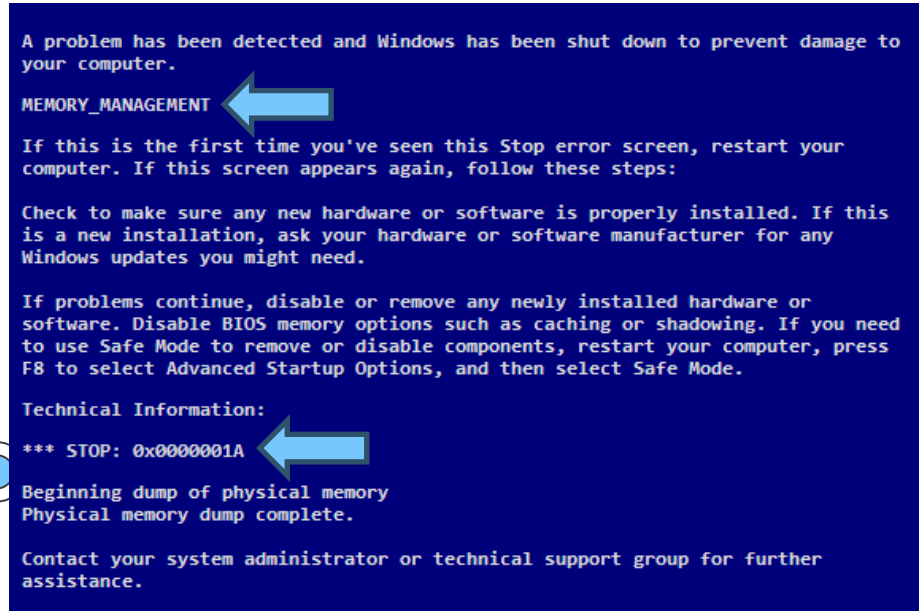




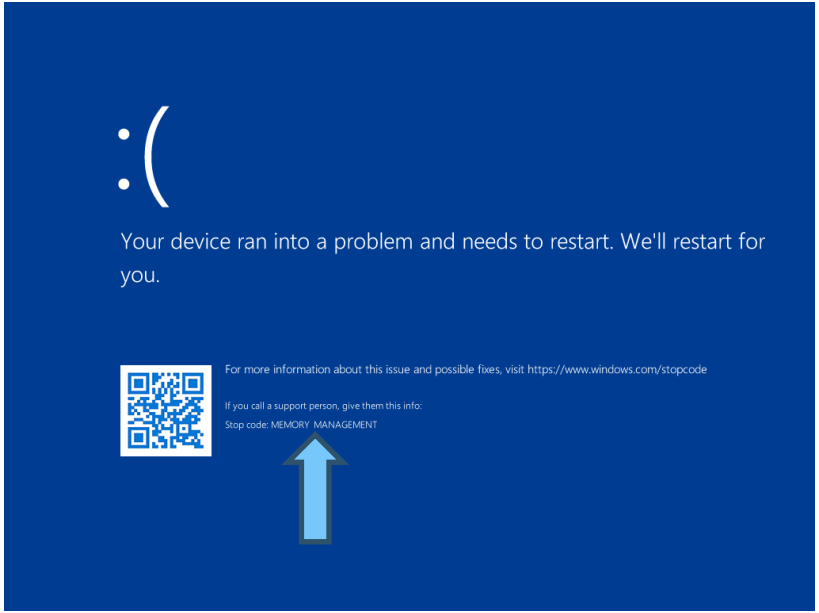
2.1 Armazenamento, constantes e variáveis

Você já deve ter visto um endereço hexadecimal de memória RAM na **tela azul da morte** do Windows:

Antigamente



Hoje

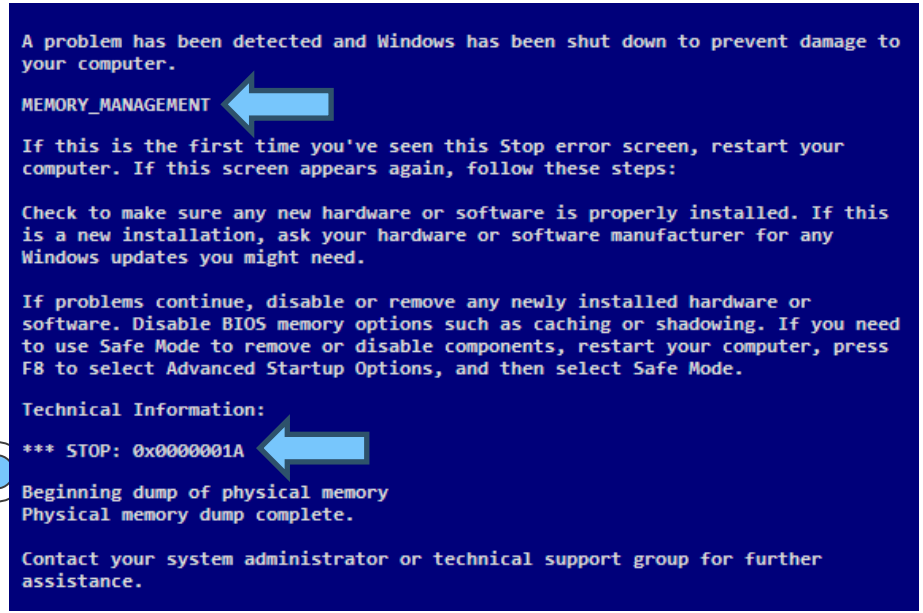


2.1 Armazenamento, constantes e variáveis

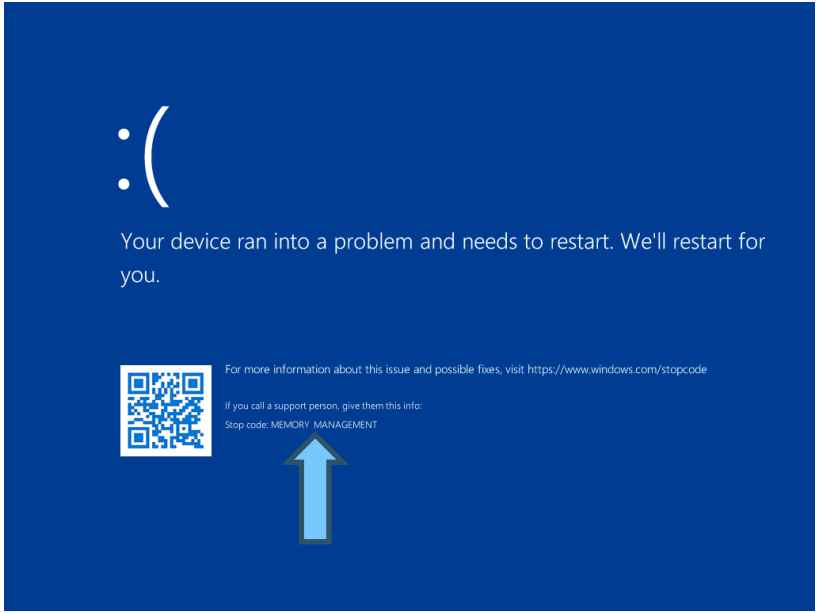
Link aqui de como resolver.

=D

Antigamente



Hoje



2.1 Armazenamento, constantes e variáveis

Os dados são representados pelas informações a serem tratadas (processadas) pelo computador.

Essas informações estão caracterizadas por três tipos de dados (ou tipos primitivos):

- Dados Numéricos (real e inteiro);
- Dados caracteres (literal);
- Dados Lógicos (falso e verdadeiro).

2.1 Armazenamento, constantes e variáveis

Tipos básicos da linguagem C:

char – caractere

int – número inteiro

float – número real de precisão simples

double – número real de precisão dupla

2.1 Armazenamento, constantes e variáveis

Dados numéricos tipo **Inteiro**

São aqueles que não possuem números fracionários ou decimais.

São caracterizados como tipos inteiros os dados numéricos positivos, negativos e nulos, excluindo-se destes qualquer número fracionário.

Exemplos:

35
-45
0

2.1 Armazenamento, constantes e variáveis

Dados numéricos tipo **Real**

São aqueles que podem possuir componentes decimais ou fracionários;

São caracterizados como tipos reais os dados numéricos positivos, negativos, nulos e números fracionários. Exemplos:

35,00
-1,5
51,65
0,0

2.1 Armazenamento, constantes e variáveis

Dados tipo **Caractere** (Literal)

São caracterizados como tipo Literal as sequências contendo letras, números e símbolos especiais. Uma sequência de caracteres dever ser indicada entre aspas (“ ”). Este tipo de dado é também conhecido como alfanumérico, string, ou cadeia. Exemplos:

“Engenharia Civil”
“16.389.995”
“9719-9857”
“3,5”

2.1 Armazenamento, constantes e variáveis

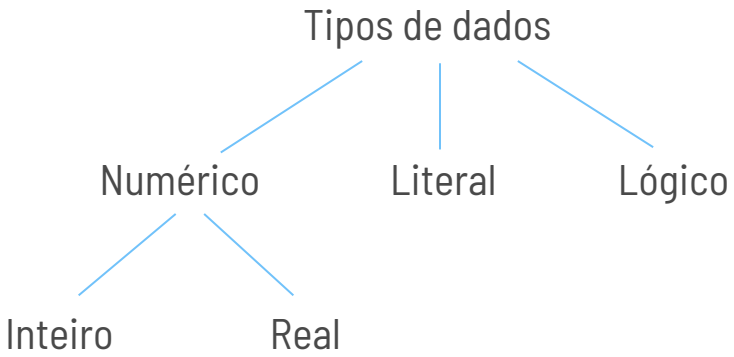
Dados tipo **Lógico**

São caracterizados como tipo Lógico os dados com valores verdadeiro e falso, sendo que este tipo de dado poderá representar apenas um dos dois valores, é também conhecido como dado Booleano.

- Em C, o tipo de dado booleano não é nativamente suportado. No entanto, é comum utilizar inteiros para representar valores booleanos, onde 0 representa "falso" e qualquer valor diferente de zero (tipicamente 1) representa "verdadeiro".

2.1 Armazenamento, constantes e variáveis

Representação gráfica dos diversos tipos de dados:



2.1 Armazenamento, constantes e variáveis

Tipos básicos da linguagem C:

Tipo	Tamanho (bytes)	Tamanho (bit)
char	1	0
int	4*	32*
float	4	43
double	8	64

* Depende do compilador

2.1 Armazenamento, constantes e variáveis

Identificadores são os nomes que damos aos objetos utilizados (variáveis, constantes, funções, etc).

Podem ter qualquer nome, desde que:

- Comecem com uma letra
- Os caracteres subsequentes devem ser letras, números ou _
- Não seja igual a: uma palavra reservada, uma função declarada ou bibliotecas do C.

2.1 Armazenamento, constantes e variáveis

Identificadores – Exemplos

Permitido	
int	_mes
int	ano_atual
float	temp1
char	palavra_digitada

Não permitido	
char	2a_feira
int	valores$
float	float
int	main
char	stdio

2.2 Transferências, atribuições, entradas e saídas

No contexto de programação em C, "**transferências**" referem-se à movimentação de dados entre diferentes áreas da memória.

Isso pode envolver a cópia de valores de uma variável para outra, transferência de dados entre funções, manipulação de arrays (listas) ou até mesmo operações mais complexas de manipulação de dados.



2.2 Transferências, atribuições, entradas e saídas

"**Atribuições**" envolvem a ação de associar ou dar um valor a uma variável.

Em C, isso é realizado usando o operador de atribuição **=**.

Por exemplo, int x = 10; atribui o valor 10 à variável x.

A atribuição é uma operação fundamental para a manipulação de dados e o controle de fluxo em programas escritos na linguagem C.



2.2 Transferências, atribuições, entradas e saídas

Uma variável, ao ser definida, contém um valor qualquer e imprevisível. Costuma-se dizer que contém **lixo de memória**.

Exemplo:
int x;

Seu valor é indefinido e pode conter dados residuais que estavam anteriormente armazenados nessa área de memória. Por isso é uma boa prática de programação inicializar as variáveis com um valor padrão (**int** x = 0).



2.2 Transferências, atribuições, entradas e saídas

Atribuição de variáveis:

```
int a = 10; // Atribuição do valor 10 à variável ‘a’ de tipo int
int b; // Declaração de uma nova variável ‘b’ de tipo int
b = a; // Transferência do valor de 'a' para ‘b’
float numeroFloat = 3.14; // Atribuição para uma variável de tipo float
double numeroDouble = 3.14159; // Atribuição para uma variável de tipo double
double numeroDouble = 3.14159; // Atribuição para uma variável de tipo double
char caractere = 'A'; // Atribuição para uma variável de tipo char
```

2.2 Transferências, atribuições, entradas e saídas

```c
/*
 ============================================================================
 Name        : Prj_StrCpy.c
 Author      :
 Version     :
 Copyright   : Your copyright notice
 Description : Hello World in C, Ansi-style
 ============================================================================
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>  // Necessário para funções relacionadas a strings

int main() {
    char string1[] = "Hello"; // Atribuição para um array de char (string)
    char string2[10];         // Declaração de outro array de char (string)
    strcpy(string2, "World"); // Atribuição usando strcpy

    printf("String1: %s\n", string1);
    printf("String2: %s\n", string2);

    return 0;
}
```

Console × Problems Tasks Properties

<terminated> (exit value: 0) Prj_StrCpy [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_StrCpy/Debug/Prj_StrCpy (11/02/2024 12:58)

```
String1: Hello
String2: World
```

2.2 Transferências, atribuições, entradas e saídas

O operador += é um operador de atribuição combinada, utilizado para incrementar o valor de uma variável com um valor específico. A expressão x += y é equivalente a x = x + y.

O mesmo princípio se aplica a outros operadores aritméticos, como -= (subtração), *= (multiplicação), /= (divisão) e %= (resto da divisão).

```
// Incrementa o valor da variável "numero" em 3
numero += 3;
```

2.2 Transferências, atribuições, entradas e saídas

Biblioteca string.h:

Em C, a biblioteca string.h é uma biblioteca padrão que fornece diversas funções para manipulação de strings (sequências de caracteres). Essas funções incluem operações para cópia, concatenação, comparação, busca e outras manipulações comuns em strings.

Ao incluir string.h, você têm acesso a essas funcionalidades, o que simplifica significativamente a manipulação de strings em C. Essa biblioteca é essencial em cenários onde a manipulação de texto é uma parte importante da lógica do programa, como na entrada/saída de dados ou na manipulação de strings de texto.

2.2 Transferências, atribuições, entradas e saídas

Função strcpy:

strcpy é uma função específica da biblioteca string.h que é usada para copiar uma string para outra. Seu nome deriva de "string copy". A função strcpy aceita dois argumentos: o primeiro é o destino, onde a string será copiada, e o segundo é a origem, de onde a string será copiada.

Ela copia os caracteres da origem para o destino até encontrar o caractere nulo ('\0'), que indica o final da string. A utilização adequada de strcpy é essencial para evitar problemas de buffer overflow, garantindo que a string de destino tenha espaço suficiente para a cópia da string de origem.

2.2 Transferências, atribuições, entradas e saídas

Função strlen:

strlen é uma função específica da biblioteca string.h em C que é utilizada para calcular o comprimento de uma string. O nome "strlen" deriva de "string length". A função strlen recebe como argumento uma string e retorna o número de caracteres presentes nessa string, excluindo o caractere nulo de terminação ('\0').

A função strlen percorre os caracteres da string, um por um, até encontrar o caractere nulo de terminação ('\0'). O comprimento retornado representa a quantidade de caracteres antes do caractere nulo.

2.2 Transferências, atribuições, entradas e saídas

Vejamos alguns caracteres de **escape** e de **controle** em C:

\0 (Barra invertida seguida de zero): Em C, o caractere nulo, representado por \0, é um caractere especial usado para indicar o final de uma string. Em C, as strings são arrays de caracteres e são terminadas por um caractere nulo;

\n (Barra invertida seguida de n): O caractere de escape \n é usado para representar uma nova linha em muitas linguagens de programação, incluindo C. Quando utilizado em uma string ou em uma instrução de impressão (como printf), ele adiciona uma quebra de linha;

**** (Barra invertida): Representa uma barra invertida literal. É usado quando você precisa imprimir uma barra invertida em sua saída: printf("C:\\Diretorio\\Arquivo");

2.2 Transferências, atribuições, entradas e saídas

Vejamos alguns caracteres de **escape** e de **controle** em C:

\" (Aspas duplas): Representa uma aspa dupla literal. É usado quando você precisa imprimir aspas duplas dentro de uma string: printf("Isso é uma \"string\"");

\' (Aspas simples): Representa uma aspa simples literal. É usado quando você precisa imprimir uma aspa simples dentro de uma string ou caractere: printf("Isso é uma \'caractere\'");

\t (Tabulação): Representa um caractere de tabulação. Geralmente, é usado para criar espaçamento horizontal: printf("Texto\tTabulado");

\r (Retorno de carro): Move o cursor para o início da linha, muitas vezes usado em conjunto com \n para representar uma nova linha: printf("Texto1\rTexto2\n");

2.2 Transferências, atribuições, entradas e saídas

I/O (Input/Output):

Comandos de entrada e saída estão descritos na biblioteca **stdio.h**.

Funções de **Entrada**:
scanf()
gets()
getchar()

Funções de **Saída**:
printf()
puts()
putchar()

2.2 Transferências, atribuições, entradas e saídas

I/O (Input/Output):

Entradas:

Operações de "entradas" referem-se à aquisição de dados do usuário ou de outras fontes externas durante a execução do programa.

Em C, a função **scanf** é comumente usada para receber entradas do teclado, enquanto em ambientes mais avançados, como em sistemas embarcados, as entradas podem vir de sensores ou outras interfaces.

2.2 Transferências, atribuições, entradas e saídas

I/O (Input/Output):

Entradas:

scanf é a rotina de leitura formatada de dados inseridos pelo usuário via teclado.

scanf(*string_de_controle*, *argumentos*);

string_de_controle:

- comandos de formato (%_), os mesmos relacionados para o printf().

argumentos:

- endereço das variáveis onde serão colocados os valores de entrada: &nome_variável.
  - O operador & representa o endereço de uma variável.

2.2 Transferências, atribuições, entradas e saídas

Exemplo de entrada com **scanf**:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Declaração da variável para armazenar o valor lido
    int numero;

    // Utilização da scanf para ler um inteiro
    printf("Digite um número inteiro: ");
    scanf("%d", &numero);

    // Exibição do valor lido
    printf("Você digitou: %d\n", numero);

    return 0;
}
```

Console | Problems | Tasks | Properties

```
<terminated> (exit value: 0) Prj_LeituraDados [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_LeituraDados/Debug/
Digite um número inteiro: 1
Você digitou: 1
```

2.2 Transferências, atribuições, entradas e saídas

I/O (Input/Output):

Saídas:

Operações de "saídas" envolvem a exibição de dados ou resultados para o usuário ou para outras partes do sistema.

Em C, a função **printf** é frequentemente utilizada para exibir informações na tela, mas em diferentes contextos, como sistemas embarcados, as saídas podem ser redirecionadas para outros dispositivos.

2.2 Transferências, atribuições, entradas e saídas

I/O (Input/Output):

Saídas:

printf é a função para a escrita formatada no dispositivo de saída padrão, normalmente o vídeo.

printf(*string_de_controle*, *argumentos*);

string_de_controle consiste em:

- Caracteres que serão exibidos na tela
- Caracteres de controle (\)
- Comandos de formato, que definem a maneira como os argumentos serão mostrados (%)

...

argumentos são as variáveis cujos conteúdos serão exibidos na tela.

2.2 Transferências, atribuições, entradas e saídas

I/O (Input/Output):

Saídas:

Alguns comandos de formato disponíveis (começam por %):

%d (%i) – números inteiros em decimal

%f – números reais do tipo float

%lf – números reais do tipo double

%c – caractere

%s – sequência de caracteres (string)

%% – o próprio caractere %

2.2 Transferências, atribuições, entradas e saídas

I/O (Input/Output):

Saídas:

A função **puts** em é utilizada para imprimir uma string (sequência de caracteres) seguida de uma nova linha (\n) no console. Ela é parte da biblioteca padrão do C e é uma forma simplificada de realizar a operação de impressão de strings, comparada com a função printf.

Ao contrário da função printf, puts não permite formatação complexa ou a inclusão de variáveis diretamente na string. Ela é mais simples e adequada para imprimir strings básicas. O \n é automaticamente adicionado pelo puts.

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

O termo "**manipulações**" em C refere-se às ações que você pode realizar em dados, variáveis ou estruturas para modificá-los de acordo com as necessidades do seu programa.

Isso pode incluir operações aritméticas, lógicas, relacionais, bem como manipulações de strings e conversões entre tipos de dados.



2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

O conceito **Expressão** em termos computacionais está diretamente ligado ao conceito de expressão (fórmula) matemática, onde um conjunto de variáveis e constantes numéricas relaciona-se por meio de operadores aritméticos compondo uma fórmula que, uma vez avaliada, resulta num valor.

As expressões dividem-se em: Aritméticas; Funções matemáticas; Lógicas e Relacionais.



2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

As "**conversões**" referem-se à mudança do tipo de dado de uma variável para outro.

Existem dois tipos principais de conversões em C:

conversões implícitas ou coerções (automáticas);

e conversões explícitas (cast).



2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Conversões implícitas ou coerções:

As conversões implícitas ocorrem automaticamente pelo compilador quando é seguro fazê-lo sem perda de dados.

Isso geralmente acontece em operações onde os operandos têm tipos diferentes, mas a linguagem permite a conversão de um tipo para outro de maneira segura.

```
int inteiro = 13;
float decimal = inteiro;  // Conversão implícita de int para float
```



2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Conversões explícitas (cast):

As conversões explícitas, ou cast, são realizadas explicitamente pelo programador.

Isso é feito colocando o tipo desejado entre parênteses antes da variável que precisa ser convertida.

```
float decimal = 3.14;
int inteiro = (int)decimal; // Conversão explícita de float para int
```



2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Conversões explícitas (cast):

Aqui, (int) é um cast que indica ao compilador para converter decimal de float para int.

No entanto, este tipo de conversão pode resultar em perda de dados se a parte decimal não puder ser representada como um número inteiro.

```
float decimal = 3.14;
int inteiro = (int)decimal;  // Conversão explícita de float para int
```



2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Conversões de caracteres:

Você pode fazer conversões entre tipos de caracteres, como de char para int ou vice-versa.

```
char caractere = 'A';
int valorAscii = (int)caractere; // Conversão de char para int
```

Consulte:

https://www.matematica.pt/docs/diversos/resumos/tabela-ascii.pdf



2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores da Linguagem C:

	Operador	Tipo
Operador unário	++, --	Incremento/Decremento
Operador binário	+, -, *, /, %	Aritméticos
	<, <=, >, >=, ==, !=	Relacionais
	&&, ||, !	Lógicos
	&, |, <<, >>, ~, ^	Bitwise
	=, +=, -=, *=, /=, %=	Atribuição
Operador ternário	?:	Condicional

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores Aritméticos:

Operador	Ação	Exemplo
+	Adição	x = x + y
-	Subtração	x = x - y
*	Multiplicação	x = 8 * y
/	Divisão	x = y / 2
%	Resto da divisão	x = y % 2
++	Incremento	x++
--	Decremento	y--

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores Aritméticos:

Diversos operadores podem ser utilizados em uma mesma expressão:

```
double area = 2 * PI * raio;
double delta = b * b - 4 * a * c;
```

A ordem de execução dos operadores pode ser determinada explicitamente pelo uso de parênteses:

```
double media = (a + b + c) / 2;
double delta = (b*b) - (4*a*c);
```

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Quando não há parênteses, o compilador usa regras de precedência para determinar qual operação executar primeiro.

Maior precedência
- (unário)
* / %
+ -
Menor precedência

Se duas operações têm a mesma precedência, segue-se a ordem da esquerda para a direita.

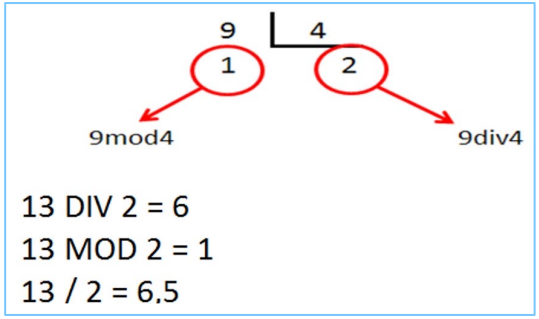
2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Expressões **aritméticas** são aquelas cujo resultado da avaliação é do tipo numérico, seja ele Inteiro ou Real. Somente o uso de operadores aritméticos e variáveis numéricas é permitido em expressões deste tipo.

Tanto variáveis como constantes poderão ser utilizadas na elaboração de expressões aritméticas.

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Vejamos outras operações matemáticas não convencionais cujos nomes dos operadores são:



MOD = Resto da divisão

DIV = Quociente da divisão

Atenção: Na linguagem de programação C o operador MOD é representando pelo símbolo %.

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Além das operações básicas anteriormente citadas, podemos usar nas expressões aritméticas algumas funções matemáticas:

Sen(X), Cos(X), Abs(X), Int(X)

Existem muitas outras funções matemáticas disponíveis, onde X pode ser uma constante numérica, variável, expressão aritmética ou também outra função matemática.

- Lembre-se de incluir as bibliotecas apropriadas (como <math.h> e <stdlib.h>) para usar essas funções em C.

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores Relacionais:

Uma expressão relacional é uma comparação realizada entre dois valores de mesmo tipo básico. Esses valores são representados na relação através de constantes, variáveis ou expressões aritméticas. O resultado de uma comparação é sempre um valor **lógico**.

Operador	Descrição
<	Menor que
<=	Menor ou igual
>	Maior que
>=	Maior ou igual
==	Igual
!=	Diferente

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores Relacionais:

Exemplo 1:

```
var = 2;
if (var>2)
var = 3;
```

- O valor de var continua 2.
- Um erro comum é confundir o operador de comparação (==) com o operador de igualdade (=)

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores Relacionais:

Exemplo 2:

```
var = 1;
if (var=2)
var = 3;
```

- O valor de var será 3, o que certamente não era esperado. Isso ocorre porque var=2 atribui 2 a var e, desde que foi executado, a expressão retornará um valor verdadeiro para a declaração if, permitindo a execução da linha seguinte.
- Portanto, o correto seria: if(var==2)

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores Lógicos:

Operam sobre os valores lógicos das variáveis.

Operador	Ação
&&	AND (E)
||	OR (OU)
!	NOT (NÃO)

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores Lógicos:

Usados normalmente com expressões booleanas, isto é, expressões que retornam verdadeiro ou falso (1 ou 0), para fins de testes em declarações condicionais.

Operador	Função
E	CONJUNÇÃO
OU	DISJUNÇÃO
NÃO	NEGAÇÃO

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Operadores Lógicos:

TABELA VERDADE					
A	B	A e B	A ou B	Não A	Não B
V	V	V	V	F	F
V	F	F	V	F	V
F	V	F	V	V	F
F	F	F	F	V	V

Exemplos:

2 < 5	ou	15 / 3 = 5
V	ou	V
	V	

2 < 5	e	15 / 3 = 4
V	e	F
	F	

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

&& (E lógico) Retorna verdadeiro se ambos os operandos são verdadeiros e falso nos demais casos.

Exemplo: if(a>3 && b<4).

|| (OU lógico) Retorna verdadeiro se um ou ambos os operandos são verdadeiros e falso se ambos são falsos.

Exemplo: if(a>3 || b<4).

! (NÃO lógico) Usada com apenas um operando. Retorna verdadeiro se ele é falso e vice-versa.

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

O operador **ternário** pode ser útil em casos onde você deseje evitar escrever if's e else's.

Estrutura:

condição ? expressão_se_verdadeira : expressão_se_falsa;

```
int numero = 10; // Estrutura de "se senão" usando operador ternário
int resultado = (numero > 5) ? 2 : 3;
```

Neste exemplo, se numero for maior que 5, a variável resultado receberá o valor 2; caso contrário, receberá o valor 3.

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Veja alguns códigos que fiz, em uma linha, utilizando o operador ternário:

https://github.com/joaopauloaramuni/c/blob/main/Aramuni.c

(Mais adiante, veremos recursividade em detalhes.)

```c
int fatorial(int c) {
return (c == 0 || c == 1) ? 1 : (c * fatorial(c - 1));
}

int fibonacci(int n) {
return (n == 0) ? 0: ((n == 1) ? 1 : (fibonacci(n - 1) + fibonacci(n - 2)));
}
```

2.3 Manipulações, conversões, operações (aritméticas, lógicas, relacionais e literais) e operadores

Dica: Caso você esteja fora do seu computador pessoal, ou sem sua IDE - *Integrated Development Environment* (como eclipse e VS Code), utilize compiladores disponíveis online:

- https://www.programiz.com/c-programming/online-compiler/
- https://ideone.com/
- https://www.onlinegdb.com/online_c_compiler

Canais/vídeos sugeridos:

- https://www.youtube.com/@progdescomplicada/
- https://www.youtube.com/@BenjaminGrandoMoreira

Leitura sugerida:

- https://www.galirows.com.br/meublog/programacao/operadores-bitwise-cpp/

2.4 Estruturas de controle

As **estruturas de controle** são utilizadas para alterar o fluxo de execução do programa.

Elas permitem que você tome decisões, repita a execução de blocos de código ou execute diferentes partes do código com base em condições específicas.

Vejamos as principais estruturas de controle em C.



2.4.1 Estrutura sequencial

A estrutura sequencial refere-se à execução linear de instruções em um programa de computador.

Em outras palavras, as instruções são executadas em ordem, uma após a outra, de cima para baixo, sem desvios condicionais ou repetições durante a execução dessa sequência específica.



2.4.1 Estrutura sequencial

Em linguagens de programação como C, a estrutura sequencial é a forma mais básica de organização do código.

As instruções são executadas em uma ordem predefinida, e o fluxo de controle não é alterado até que todas as instruções da sequência sejam executadas.

```c
#include <stdio.h>

int main(void) {
    // Instruções são executadas sequencialmente
    printf("Hello, ");  // Exibe "Hello, "
    printf("world!");   // Exibe "world!"

    return 0;
}
```

2.4.1 Estrutura sequencial

Para praticar, vamos criar um algoritmo em três etapas:

I) Criação de um fluxograma

II) Criação de um pseudocódigo

III) Criação do código em C

(Também por isso é importante ter um caderno para a disciplina de Algoritmos e Estruturas de Dados. As etapas I e II podem ser feitas no papel.)

2.4.1 Estrutura sequencial

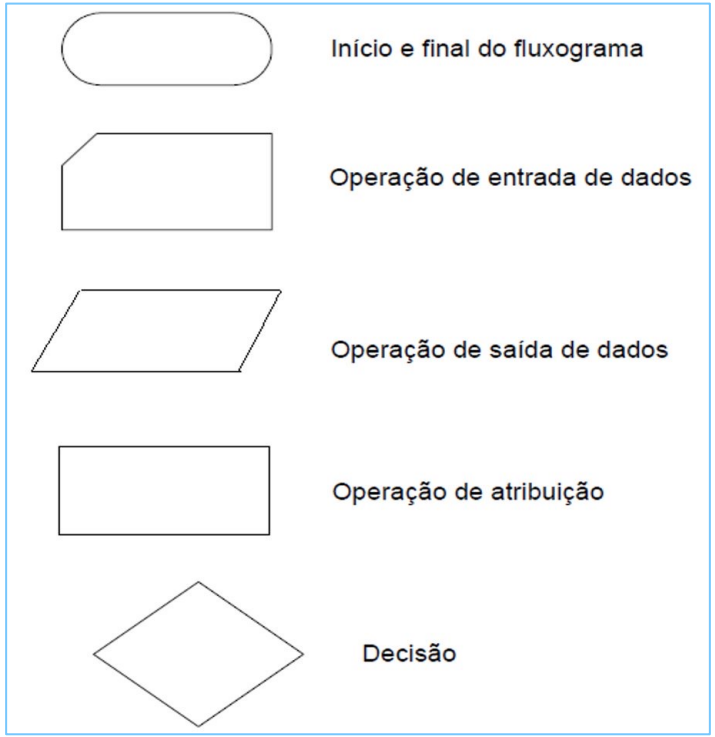
Fluxograma:

É uma representação gráfica de algoritmos onde formas geométricas diferentes implicam ações (instruções, comandos) distintos. Tal propriedade facilita o entendimento das ideias contidas nos algoritmos e justifica sua popularidade.

Esta forma é aproximadamente intermediária à descrição narrativa e ao pseudocódigo, pois é menos imprecisa que a primeira e, no entanto, não se preocupa com detalhes de implementação do programa, como o tipo das variáveis usadas.

2.4.1 Estrutura sequencial

Fluxograma:

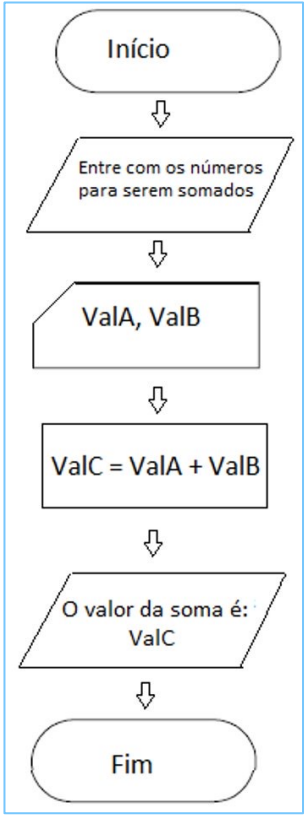


2.4.1 Estrutura sequencial

Estrutura sequencial:

I) Fluxograma

Cada comando no código é executado sequencialmente, na ordem em que são escritos.



2.4.1 Estrutura sequencial

Estrutura sequencial:

II) Pseudocódigo

```
Início
inteiro: ValA, ValB, ValC;
escreva("Entre com os números para serem somados");
leia(ValA);
leia(ValB);
ValC ← ValA + ValB;
escreva ("O valor da soma é: ", ValC);
Fim.
```

2.4.1 Estrutura sequencial

Estrutura sequencial:

III) Código em C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int ValA, ValB, ValC;

    printf("Entre com os números para serem somados: ");
    scanf("%d", &ValA);
    scanf("%d", &ValB);

    ValC = ValA + ValB;

    printf("O valor da soma é: %d\n", ValC);

    return 0;
}
```

2.4.2 Estruturas condicionais: simples, composta e múltipla

As estruturas condicionais são elementos fundamentais em linguagens de programação que permitem a execução condicional de blocos de código.

Elas são cruciais para criar programas dinâmicos e flexíveis, capazes de tomar decisões com base em condições específicas. As estruturas condicionais mais comuns são if, else if, else, switch e case.

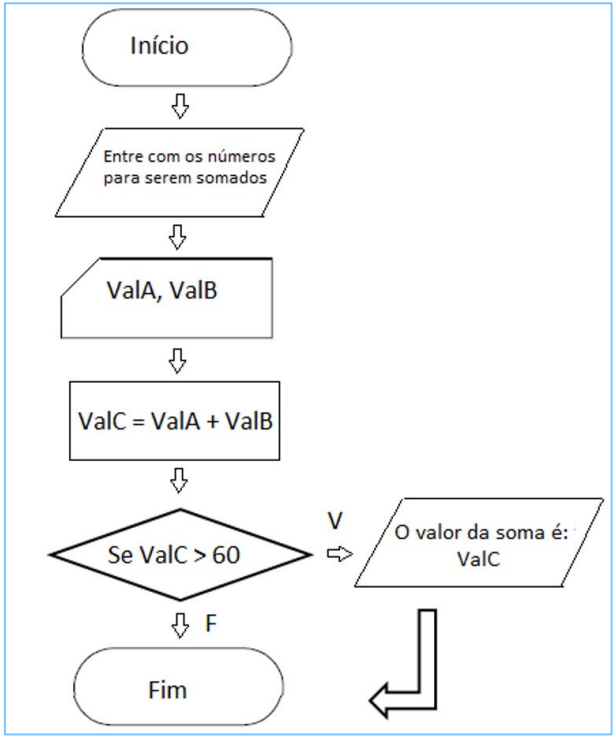


2.4.2 Estruturas condicionais: simples, composta e múltipla

Estrutura condicional **simples**:

I) Fluxograma

Permite a escolha de um grupo de ações (bloco), quando certas condições são (ou não são) satisfeitas. Utilizada quando precisamos testar uma certa condição antes de executar uma ação.



2.4.2 Estruturas condicionais: simples, composta e múltipla

Estrutura condicional **simples**:

II) Pseudocódigo

```
Início
inteiro: ValA, ValB, ValC;
escreva("Entre com os números para serem somados");
leia(ValA);
leia(ValB); ValC ← ValA + ValB;
Se ValC > 60 entao
    escreva ("O valor da soma é: ", ValC);
FimSe;
Fim.
```

2.4.2 Estruturas condicionais: simples, composta e múltipla

Estrutura condicional **simples**:

III) Código em C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int ValA, ValB, ValC;

    printf("Entre com os números para serem somados: ");
    scanf("%d", &ValA);
    scanf("%d", &ValB);

    ValC = ValA + ValB;

    if (ValC > 60) {
        printf("O valor da soma é: %d\n", ValC);
    }

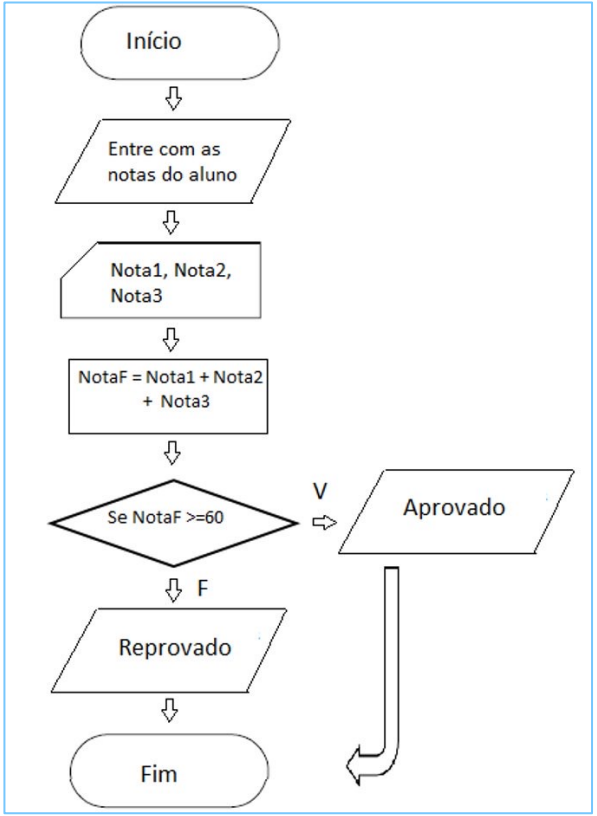
    return 0;
}
```

2.4.2 Estruturas condicionais: simples, composta e múltipla

Estrutura condicional **composta**:

I) Fluxograma

Permite a escolha de um grupo de ações (bloco), quando certas condições são (ou não são) satisfeitas. Utilizada em situações em que duas alternativas dependem da mesma condição, uma da condição verdadeira (então) e a outra da condição falsa (senão).



2.4.2 Estruturas condicionais: simples, composta e múltipla

Estrutura condicional **composta**:

II) Pseudocódigo

```
Início
real: Nota1, Nota2, Nota3, NotaF;
escreva("Entre com as notas do aluno");
leia(Nota1, Nota2, Nota3);
NotaF ← Nota1 + Nota2 + Nota3;
Se NotaF >= 60 entao
    escreva ("Aprovado");
Senao
    escreva ("Reprovado");
FimSe;
Fim.
```

2.4.2 Estruturas condicionais: simples, composta e múltipla

Estrutura condicional **composta**:

III) Código em C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    float Nota1, Nota2, Nota3, NotaF;

    printf("Entre com as notas do aluno: ");
    scanf("%f %f %f", &Nota1, &Nota2, &Nota3);

    NotaF = Nota1 + Nota2 + Nota3;

    if (NotaF >= 60) {
        printf("Aprovado\n");
    } else {
        printf("Reprovado\n");
    }

    return 0;
}
```

2.4.2 Estruturas condicionais: simples, composta e múltipla

Estrutura condicional **múltipla**:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int escolha;

    printf("Digite um número de 1 a 5: ");
    scanf("%d", &escolha);

    if (escolha == 1) {
        printf("Você escolheu o número 1.\n");
    } else if (escolha == 2) {
        printf("Você escolheu o número 2.\n");
    } else if (escolha == 3) {
        printf("Você escolheu o número 3.\n");
    } else if (escolha == 4) {
        printf("Você escolheu o número 4.\n");
    } else if (escolha == 5) {
        printf("Você escolheu o número 5.\n");
    } else {
        printf("Opção inválida.\n");
    }
```

2.4.2 Estruturas condicionais: simples, composta e múltipla

Estrutura condicional **múltipla**:

```c
// Alternativa mais enxuta
switch (escolha) {
case 1:
    printf("Você escolheu o número 1.\n");
    break;
case 2:
    printf("Você escolheu o número 2.\n");
    break;
case 3:
    printf("Você escolheu o número 3.\n");
    break;
case 4:
    printf("Você escolheu o número 4.\n");
    break;
case 5:
    printf("Você escolheu o número 5.\n");
    break;
default:
    printf("Opção inválida.\n");
}

return 0;
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

As estruturas de repetição desempenham um papel crucial no desenvolvimento de programas eficientes e dinâmicos, permitindo que instruções sejam executadas repetidamente enquanto uma condição específica é atendida.

Essa capacidade de automação e iteração é fundamental para lidar com tarefas que envolvem a manipulação de conjuntos de dados extensos, validação de entradas do usuário, ou a execução de operações em larga escala.



2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

Além de otimizar o código, as estruturas de repetição contribuem para a legibilidade e manutenção do software, uma vez que permitem encapsular lógicas recorrentes em blocos coesos.

Dessa forma, asseguram que programas possam lidar eficazmente com uma variedade de cenários, promovendo a eficiência, modularidade e flexibilidade no desenvolvimento de software.



2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

É comum situações em que se deseja repetir um determinado trecho de um programa certo número de vezes.

Por exemplo, quando se deseja realizar um mesmo processamento para conjuntos de dados diferentes.

Exemplo: processamento de folha de pagamentos de uma empresa em que o mesmo cálculo é efetuado para cada um dos funcionários.



2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

As estruturas de repetição são também denominadas Laços (Loops).

Baseado no conhecimento prévio do número de vezes que o conjunto de instruções será executado, as estruturas de repetição podem ser classificadas em:

- **Laços contados**: quando se conhece previamente quantas vezes o comando composto no interior da construção será executado;
- **Laços condicionais**: quando não se conhece o número de vezes que o conjunto de instruções no interior do laço será repetido.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

Contador:

Pode ser também uma variável, inteira, que armazene a quantidade de números digitados positivos, número de pessoas, etc.

Pode ser uma variável de controle, inteira, que serve para controlar quantas vezes um determinado trecho de programa será executado.

Ex: cont = cont + 1;

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

Acumulador:

Pode ser também uma variável, inteira ou real que acumula valores.

Acumula/soma salários:

SomaSal = 0;

....

SomaSal = SomaSal + Salario;

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

Acumulador:

Pode ser também uma variável, inteira ou real que acumula valores.

Acumula idades:

TotIdade = 0;

....

TotIdade = TotIdade + Idade;

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

enquanto...faça:

Sua semântica é a seguinte: ao início da estrutura de repetição Enquanto a condição é testada. Se o resultado do teste for falso, então as instruções no seu interior não serão executadas e a execução prossegue normalmente pela instrução seguinte ao fimenquanto.

Se a condição for verdadeira as instruções serão executadas e ao seu término retorna-se ao teste da condição. Assim, o processo acima será repetido enquanto a condição testada for verdadeira.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

enquanto...faça:

Para o pseudocódigo, utilizamos as palavras reservadas enquanto, faca e fimenquanto para formar o laço de repetição.

Forma geral:

```
enquanto (condição) faca
    // comandos
fimenquanto;
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

enquanto...faça:

Exemplo:

Faça um algoritmo que receba dois números, faça a soma destes dois números e apresente o resultado.

Este algoritmo deverá ser executado 3 vezes sem sair.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

enquanto...faça:

```
Início
inteiro: x, y, result, cont;
cont = 1;
enquanto (cont <= 3) faca
    escreva("Entre com os dois números para serem somados");
    leia(x,y);
    result = x+y;
    escreva("O resultado da soma eh:", result);
    cont = cont+1;
fimenquanto;
Fim.
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

while:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int x, y, result, cont;
    cont = 1;

    while (cont <= 3) {
        printf("Entre com os dois números para serem somados: ");
        scanf("%d %d", &x, &y);

        result = x + y;

        printf("O resultado da soma é: %d\n", result);

        cont = cont + 1;
    }

    return 0;
}
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

while infinito:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {

    while (1) {
        printf("Exemplo de loop infinito.\n");
    }

    return 0;
}
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

para...faça:

Este tipo de estrutura é útil quando se conhece previamente o número de vezes que se deseja executar um determinado conjunto de comandos.

Este tipo de laço nada mais é que uma estrutura dotada de mecanismos para contar o número de vezes que o corpo do laço é executado. Nesta estrutura, a variável de controle é inicializada com um valor junto ao comando Para. Para cada iteração, seu valor é comparado com um valor final.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

para...faça:

A execução da lista de instruções se repete até que o valor armazenado na variável de controle seja maior que o valor final.

Para cada iteração, a variável de controle é incrementada (+1);

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

para...faça:

Para o pseudocódigo, utilizamos as palavras reservadas para, ate, passo

e faca para formar o laço de repetição.

Forma geral:

```
para (variavel de valor_ini ate valor_fim passo 1) faca
   // comandos
fimpara;

para (x de 1 ate 3 passo 1) faca
   escreva("Testando o comando Para... faca");
fimpara;
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

para...faça:

Exemplo:

Faça um algoritmo que receba dois números, faça a soma destes dois números e apresente o resultado.

Este algoritmo deverá ser executado 3 vezes sem sair.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

para...faça:

```
Início
inteiro: x, y, result, cont;

para (cont de 1 ate 3 passo 1) faca
    escreva("Entre com os dois números para serem somados");
    leia(x,y);
    result = x+y;
    escreva("O resultado da soma eh:", result);
fimpara;

Fim.
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

para...faça:

```
int main(void) {
    int x, y, result, cont;

    for (cont = 1; cont <= 3; cont++) {
        printf("Entre com os dois números para serem somados: ");
        scanf("%d %d", &x, &y);

        result = x + y;

        printf("O resultado da soma é: %d\n", result);
    }

    return 0;
}
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

Exercício 1:

Faça um algoritmo que leia X números, faça a leitura dos números e imprima quantos números são pares e quantos números são impares. O algoritmo pára quando o usuário digitar 0.

Exercício 2:

Faça um algoritmo que leia X números, faça a contagem dos números e imprima a mesma. O algoritmo pára quando o usuário digitar -1.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

```
Início
Inteiro: contpar, contimpar, N1;
contpar = 0; contimpar = 0; num = 400;
enquanto (num != 0) faca
    escreva( "Entre com um número para encerrar digite 0");
    leia(num);
    Se (num != 0) entao
        Se (num MOD 2 =0) entao
            contpar = contpar+1;
        senao
            contimpar = contimpar+1;
        Fimse;
    Fimse;
fimenquanto;
escreva( "A quantidade de números pares é: ", contpar);
escreva( "A quantidade de números impares é: ", contimpar);
Fim.
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int contpar = 0, contimpar = 0, num = 400;

    while (num != 0) {
        printf("Entre com um número; para encerrar, digite 0: ");
        scanf("%d", &num);

        if (num != 0) {
            if (num % 2 == 0) {
                contpar = contpar + 1;
            } else {
                contimpar = contimpar + 1;
            }
        }
    }

    printf("A quantidade de números pares é: %d\n", contpar);
    printf("A quantidade de números ímpares é: %d\n", contimpar);

    return 0;
}
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

```
Início
Inteiro: Num, contnum; num = 999; contnum = 0;
enquanto (num != -1) faca
    escreva( "Entre com um número para encerrar digite -1");
    Leia(num);
    Se (num != -1) entao
        contnum = contnum +1;
    Fimse;
Fimenquanto;
escreva( "A quantidade de números digitados é: ", contnum);
Fim.
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int Num = 999, contnum = 0;

    while (Num != -1) {
        printf("Entre com um número; para encerrar, digite -1: ");
        scanf("%d", &Num);

        if (Num != -1) {
            contnum = contnum + 1; // ou contnum++
        }
    }

    printf("A quantidade de números digitados é: %d\n", contnum);

    return 0;
}
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

faça...enquanto:

(teste no final)

A estrutura de controle do-while em muitas linguagens de programação, incluindo C, é semelhante ao while, mas garante que o bloco de código dentro do loop seja executado pelo menos uma vez, mesmo que a condição inicial seja falsa.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

faça...enquanto:

(teste no final)

Isso é útil em situações em que você deseja garantir que um bloco de código seja executado pelo menos uma vez, independentemente da condição de teste inicial. Um exemplo comum de uso do do-while é em menu de usuário, onde você deseja que o usuário escolha uma opção antes de sair do menu.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

```c
int main(void) {
    int escolha;

    do {
        printf("Menu:\n");
        printf("1. Opção 1\n");
        printf("2. Opção 2\n");
        printf("3. Sair\n");

        printf("Escolha uma opção: ");
        scanf("%d", &escolha);

        switch (escolha) {
            case 1:
                printf("Você escolheu a Opção 1.\n");
                break;
            case 2:
                printf("Você escolheu a Opção 2.\n");
                break;
            case 3:
                printf("Saindo do programa.\n");
                break;
            default:
                printf("Opção inválida. Tente novamente.\n");
        }

    } while (escolha != 3);

    return 0;
}
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

faça...enquanto:

(teste no final)

Exemplo:

Faça um algoritmo que receba dois números, faça a soma destes dois números e apresente o resultado.

Este algoritmo deverá ser executado 3 vezes sem sair.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

faça...enquanto:

(teste no final)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int x, y, result, cont;

    cont = 1;
    do {
        printf("Entre com os dois números para serem somados\n");
        scanf("%d %d", &x, &y);

        result = x + y;
        printf("O valor da soma é %d \n", result);
        cont++;
    } while (cont <= 3);

}
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

Resolva os exercícios anteriores utilizando **do-while**:

Exercício 1:

Faça um algoritmo que leia X números, faça a leitura dos números e imprima quantos números são pares e quantos números são impares. O algoritmo pára quando o usuário digitar 0.

Exercício 2:

Faça um algoritmo que leia X números, faça a contagem dos números e imprima a mesma. O algoritmo pára quando o usuário digitar -1.

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int contpar = 0, contimpar = 0, num;

    do {
        printf("Entre com um número; para encerrar, digite 0: ");
        scanf("%d", &num);

        if (num != 0) {
            if (num % 2 == 0) {
                contpar = contpar + 1;
            } else {
                contimpar = contimpar + 1;
            }
        }
    } while (num != 0);

    printf("A quantidade de números pares é: %d\n", contpar);
    printf("A quantidade de números ímpares é: %d\n", contimpar);

    return 0;
}
```

2.4.3 Estruturas de repetição: teste no início, final e com variável de controle

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int Num, contnum = 0;

    do {
        printf("Entre com um número; para encerrar, digite -1: ");
        scanf("%d", &Num);

        if (Num != -1) {
            contnum = contnum + 1; // ou contnum++
        }
    } while (Num != -1);

    printf("A quantidade de números digitados é: %d\n", contnum);

    return 0;
}
```

Disciplina – Algoritmos e Estruturas de Dados I

Referências básicas:

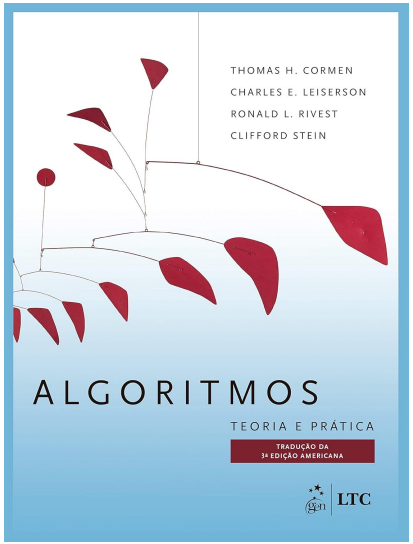
- CORMEN, Thomas. **Algoritmos** - Teoria e Prática. Editora GEN LTC. 3ª edição, 2012.

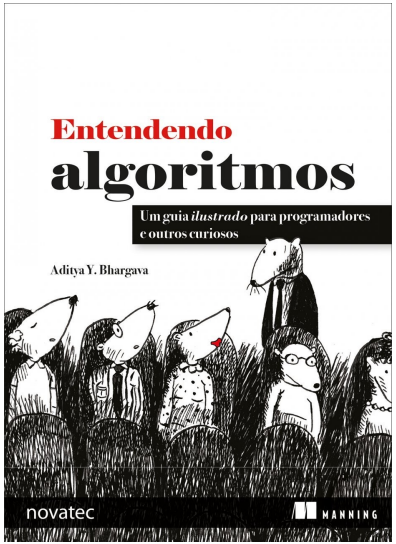
- BHARGAVA, Aditya Y. **Entendendo Algoritmos**: Um Guia Ilustrado Para Programadores e Outros Curiosos. Novatec Editora, 2017.

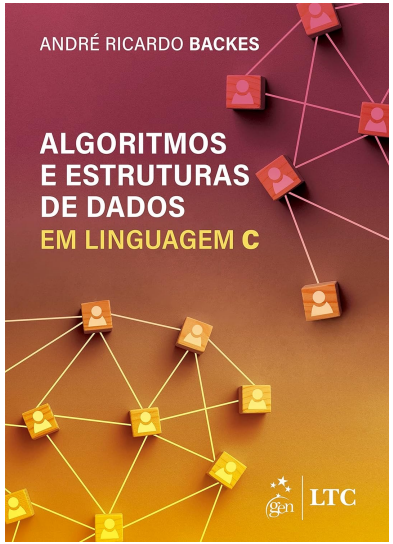
- BACKES, André Ricardo. **Algoritmos e Estruturas de Dados em Linguagem C**. Editora LTC, 2022.

Disciplina – Algoritmos e Estruturas de Dados I

Referências básicas:







Disciplina – Algoritmos e Estruturas de Dados I

Referências complementares:

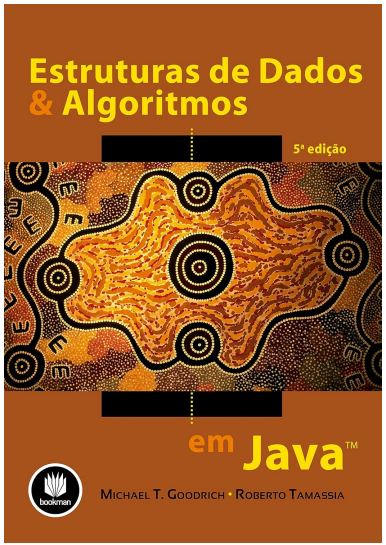
- GOODRICH, Michael T. **Estruturas de Dados e Algoritmos em Java**. Editora Bookman. 5ª edição, 2013.

- AGARWAL, Basant. **Estruturas de Dados e Algoritmos com Python**: Armazene, manipule e acesse dados de forma eficaz e melhore o desempenho de suas aplicações. Novatec Editora, 2023.

- GRONER, Loiane. **Estruturas de Dados e Algoritmos com JavaScript**: Escreva um Código JavaScript Complexo e Eficaz Usando a Mais Recente ECMAScript. Novatec Editora, 2019.

Disciplina – Algoritmos e Estruturas de Dados I

Referências complementares:







Disciplina – Algoritmos e Estruturas de Dados I

Outras referências:

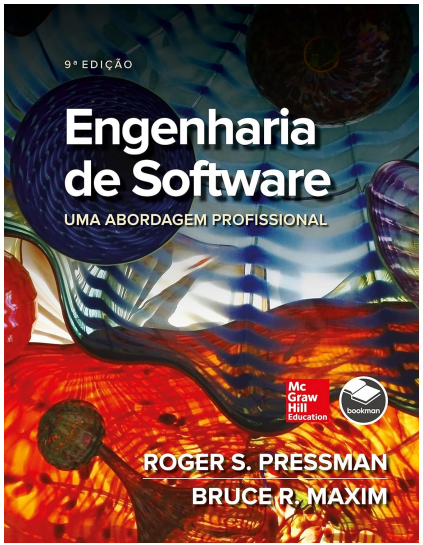
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**. Editora Grupo AMGH, 2021.

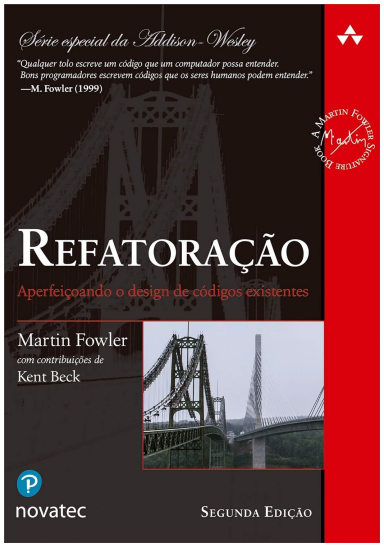
- FOWLER, Martin. **Refatoração**: Aperfeiçoando o Design de Códigos Existentes. Editora Novatec, 2020.

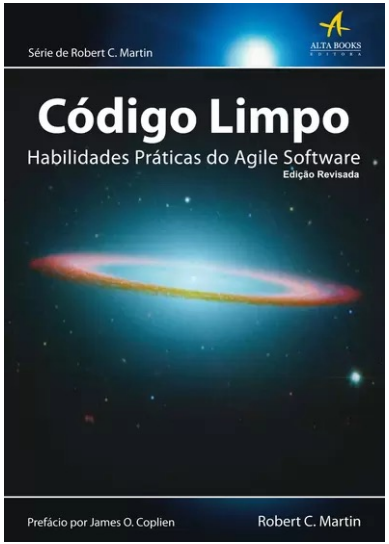
- MARTIN, Robert C. **Código limpo**: habilidades práticas do Agile software. Editora Alta Books, 2009.

Disciplina – Algoritmos e Estruturas de Dados I

Outras referências:







Disciplina – Algoritmos e Estruturas de Dados I

Nesta disciplina usaremos: Linguagem C (ANSI Style)

- Linguagem C
[DevDocs] https://devdocs.io/c/

- Eclipse IDE
[Download] https://www.eclipse.org/downloads/packages/release/2023-12/r/eclipse-ide-cc-developers
[Docs] https://www.eclipse.org/documentation/





Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com





GitHub LinkedIn Lattes